UNIVERSITY OF VERONA

DEPARTMENT OF COMPUTER SCIENCE
Master's degree in Data Science

# Dynamic User Profile Injection: A Closed-Loop RAG Architecture for Context-Aware LLMs

*Supervisor:*
Professor
**Niccolò Marastoni**

*Candidate:*
ID: VR512657
**Andrea Murari**

**ACADEMIC YEAR 2025/2026**

# Contents

# Chapter 1

# Introduction

## 1.1 Context and Motivation

The proliferation of expansive digital repositories has created an unprecedented wealth of information. However, a significant barrier persists between the availability of these complex datasets and the ability of end-users to effectively discover, combine, and extract actionable insights from them. To address this challenge, the European Union, under the Horizon Europe Research and Innovation Programme, launched the **DataGEMS** project (Grant Agreement No. 101188416) [1]. DataGEMS aims to bridge the gap between data providers and consumers by developing a next-generation dataset discovery and management ecosystem. By leveraging Natural Language Processing (NLP) and Artificial Intelligence, the project seeks to democratize data access, enabling intuitive, AI-assisted exploration and recommendation journeys across diverse, data-intensive domains.

Within this broader European initiative, the integration of generative AI into Educational Technology (EdTech) serves as a critical and highly demanding use case. Specifically, the DataGEMS project incorporates the *MathE* platform [2]—an infrastructure designed to assist higher education students in mathematics—as a primary testing ground for personalized data interaction. Deploying LLMs as educational consultants in this environment exposes a fundamental limitation in current human-computer interaction paradigms: standard foundational models are inherently stateless. While they excel at retrieving static knowledge to answer isolated queries, they lack the episodic memory required to act as effective, long-term assistants.

A traditional, stateless AI tutor evaluates each user prompt in a vacuum. It cannot recall previous misconceptions, track the gradual degradation of retained knowledge, or recognize fluctuations in the user's engagement. In complex domains like education, this static prompt-response mechanism severely limits the pedagogical value of the system. Without an underlying architecture capable of tracking the user's evolving state, the interaction devolves into repetitive, one-size-fits-all responses that fail to adapt to the learner's cognitive trajectory.

Therefore, the primary motivation for this thesis is the necessity to evolve from static, open-loop data-retrieval interfaces to dynamic, memory-aware adaptive systems. To fully realize the DataGEMS vision of personalized AI-assisted exploration, an LLM must be transformed into an intelligent agent. By engineering a closed-loop architecture capable of continuously tracking historical interactions, knowledge levels, and topical interests, this research aims to overcome the limitations of stateless AI, providing a robust proof of concept that can be seamlessly integrated into advanced ecosystems like MathE.

## 1.2 Contributions of this Thesis

By balancing selective context injection with continuous profile evaluation, the system establishes a scalable framework for highly personalized, token-efficient human-computer interactions. The core of this proposed architecture is structured around three foundational pillars, presented in the order of the system's execution pipeline:

- **Hierarchical Context Routing:** To minimize the prompt payload and reduce Information Overload on the core generative model during the retrieval phase, a multi-label classification Router is implemented. The architectural pivot from a "Flat" routing mechanism—which suffered from probabilistic retrieval failures and semantic ambiguity—to a highly accurate "Hierarchical" Router is documented. By dynamically injecting a deterministic parent-child relationship map (Topic → Subtopics), the system achieves exact context retrieval while maintaining strict token optimization.

- **Continuous State Tracking via LLM-as-a-Judge [3]:** Following the context injection and generation phases, an asynchronous evaluation module is introduced to analyze every user-system interaction in the background. Instead of relying on explicit questionnaires, this module implicitly assesses the user's domain knowledge, memory degradation (lapse), and topical interest. These metrics are continuously integrated into a persistent local database using an Exponential Moving

Average (EMA) algorithm, ensuring a smooth, noise-resistant representation of the user's evolving profile.

- **Semantic Constraining and Open Ontology:** To allow the system to spontaneously track Out-of-Domain (OOD) topics introduced by the user during the tracking phase, an Open Ontology strategy is developed. Crucially, strict semantic constraints are engineered within the Evaluator's prompt to separate pure conceptual domains (e.g., Fluid Dynamics, Personal Finance) from meta-activities and resource formats (e.g., Itinerary Planning, YouTube Channels), guaranteeing the stability and semantic relevance of the generated database.

To rigorously validate this theoretical framework, the thesis additionally provides a comprehensive empirical evaluation. A comparative analysis of three distinct RAG approaches is conducted: a Baseline Naive model (Full Database injection), the Flat Router-Based model, and the definitive Hierarchical Router-Based model. By analyzing token usage logs, system latency, and retrieval accuracy, the research empirically demonstrates the architectural trade-offs and the Return on Investment (ROI) of the hierarchical approach.

Together, the intelligent routing engine and the asynchronous evaluation module form a robust, modular proof of concept for the next generation of context-aware LLM applications. While initially validated on educational data structures, the architecture establishes a scalable foundation for any personalized AI agent required to continuously filter and adapt to an individual user's knowledge and interests.

## 1.3 Structure of the Thesis

The remainder of this thesis is organized as follows:

- **Chapter 2 (Background)** provides the theoretical foundations necessary to contextualize the research. It covers the current state of Context-Aware Large Language Models, the Retrieval-Augmented Generation (RAG) paradigm, methodologies for User State and Knowledge Tracing, and the emerging LLM-as-a-Judge [3] evaluation model.

- **Chapter 3 (Related Work)** reviews existing literature and state-of-the-art implementations, with a specific focus on Knowledge-Augmented Language Model Prompting (K-LaMP) and predictive algorithms based on continuous user log data.

- **Chapter 4 (Data Exploration and Processing)** details the methodology used to handle the datasets acting as the proof of concept for the architecture. It covers the initial Exploratory Data Analysis (EDA) of the MathE educational dataset, the mathematical formulation of generic cognitive and affective metrics, and the Semantic Discretization process required to convert numerical scores into LLM-digestible categorical labels.

- **Chapter 5 (Architectural Design and Implementation)** forms the core engineering section of this thesis. It chronicles the evolution of the system from a Naive RAG baseline to an Intelligent State-Tracking Router, detailing the implementation of the Closed-Loop architecture, the Open Ontology strategy, and the critical role of Semantic Constraining.

- **Chapter 6 (Evaluation and Results)** presents the empirical findings of the research. It evaluates the architectural performance by comparing the latency and token optimization of the three RAG iterations, supported by detailed interaction case studies that highlight the system's proactive recommendation capabilities.

- **Chapter 7 (Conclusions)** summarizes the main findings, critically discusses the strengths and limitations of the developed architecture, and proposes future directions, including the potential integration of Vector Embeddings for multi-hop retrieval in massive-scale databases.

# Chapter 2

# Background

This chapter provides the theoretical foundations required to contextualize the proposed architecture. It examines the evolution of Large Language Models toward context-aware systems, analyzes the standard Retrieval-Augmented Generation (RAG) paradigm alongside its limitations, and explores methodologies for continuous user profiling and automated semantic evaluation.

## 2.1 Context-Aware Large Language Models

Foundational Large Language Models (LLMs) [4, 5] have demonstrated unprecedented capabilities in Natural Language Processing (NLP), reasoning, and text generation. However, at their core, these models are stateless computational engines. During inference, a standard LLM possesses no episodic memory; it evaluates each prompt in isolation, relying entirely on the static parametric knowledge acquired during its pre-training phase.

This statelessness presents a critical bottleneck for personalized human-computer interaction. Without a persistent memory mechanism, models are unable to track a user's historical context, preferences, or evolving expertise. To mitigate this, the focus of AI engineering has shifted toward developing *Context-Aware Large Language Models*. Context-awareness is achieved by dynamically injecting external variables into the model's working memory (the context window) at runtime [6]. By supplying the LLM with immediate, user-specific data alongside the primary query, the model can condition its generative output to align with the unique temporal and cognitive state of the user. Nevertheless, as the volume of historical user data grows, injecting the entirety of a user's profile becomes computationally prohibitive due to strict token limits and the phenomenon of Information Overload, necessitating intelligent retrieval mechanisms.

## 2.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) [7] has emerged as the standard architectural paradigm to ground LLMs in external, dynamic data without requiring expensive model fine-tuning. In a traditional RAG pipeline, a user's query is processed by a retrieval engine—often utilizing vector embeddings and similarity search [8]—to extract relevant documents from an external database. These documents are then concatenated with the original prompt, allowing the LLM to synthesize an answer based on facts retrieved in real-time.

While highly effective for querying static knowledge bases (e.g., corporate documentation or legal archives), the standard RAG paradigm exhibits significant limitations when applied to dynamic user profiling. Traditional RAG systems are designed to retrieve *objective external facts*, not *internal subjective states*. They typically lack the logical frameworks required to understand a user's temporal progression, such as memory degradation or fluctuating interest levels. Furthermore, standard RAG architectures operate as open-loop systems: they retrieve data to answer a query but do not inherently evaluate the interaction to update the underlying database. To track a user's evolving state accurately, the RAG paradigm must be extended from a simple read-only retrieval pipeline into a closed-loop framework capable of autonomous, continuous data updates.

## 2.3 User State and Knowledge Tracing

To construct a truly context-aware system, an LLM must be paired with a robust methodology for user modeling. In computational fields, this is historically rooted in *Knowledge Tracing* (KT), a dynamic paradigm that aims to model a user's evolving mastery over specific conceptual domains based on their historical performance. Traditional frameworks, such as Bayesian Knowledge Tracing (BKT) [9] and Deep Knowledge Tracing (DKT) [10], rely on sequence-to-sequence modeling to predict the probability that a user will correctly apply a skill in future instances. While highly effective in formalized environments where user inputs consist of deterministic binary values (e.g., correct or incorrect answers on a multiple-choice test), these classical methods struggle when applied to open-ended, natural language interactions.

In modern human-computer interaction, user profiling must expand beyond purely cognitive dimensions to encompass a multidimensional state representation. This requires a dual-axis tracking approach that monitors both cognitive and affective variables:

- **Cognitive Axes (Knowledge and Memory Lapse):** Mastery cannot be treated as a static variable. Due to the biological constraints of human memory, information retention degrades over time—a phenomenon conceptualized by Ebbinghaus' forgetting curve [11]. In a digital interaction pipeline, this necessitates the tracking of a *lapse score* alongside raw domain knowledge. Tracking retention degradation allows an intelligent system to differentiate between an item that has never been understood and an item that has been mastered but requires a memory refresher. Consequently, these metrics are primarily utilized to drive *guided learning* interactions, where the system must orchestrate structured pedagogical interventions, step-by-step problem solving, or targeted concept reviews.

- **Affective Axis (Interest and Engagement):** A comprehensive user model must also quantify the user's subjective orientation toward a topic. While cognitive metrics govern skill acquisition, the affective state—captured via an explicit *interest score*—is crucial for exploratory queries and open-ended discovery [12] **(e.g., AI-assisted dataset exploration and personalized recommendation journeys, as envisioned by modern data ecosystems)**. By maintaining this metric, an interface can dynamically pivot its generative strategy, modulating proactive recommendations to match the user's curiosity and ensuring high engagement outside of strictly goal-oriented tasks.

To achieve this continuous tracking without disrupting the user experience with explicit questionnaires, the mathematical integration of new interaction data must be computationally light and resistant to statistical noise. Algorithms such as the Exponential Moving Average (EMA) are ideal for this purpose, as they allow recent interactions to update historical profiles dynamically while smoothing out anomalous data points through an adjustable smoothing factor ($\alpha$).

## 2.4 The LLM-as-a-Judge Paradigm

The necessity of capturing implicit cognitive and affective states from unstructured natural language requires a flexible yet standardized evaluation layer. Historically, automated text evaluation relied on rigid programmatic heuristics, such as keyword matching, regular expressions, or shallow sentiment analysis models. These techniques, however, are fundamentally incapable of extracting nuanced, domain-specific insights or mapping continuous semantic states from complex user queries.

To bridge this gap, the *LLM-as-a-Judge* paradigm [3] has emerged as a state-of-the-art solution for automated semantic evaluation. This framework leverages the advanced contextual understanding and zero-shot reasoning capabilities [13] of state-of-the-art Large Language Models to act as autonomous data evaluators. Instead of generating user-facing content, an evaluator LLM is deployed as a backend telemetry agent, tasked with analyzing an unstructured interaction (the user's query and the system's subsequent response) and translating it into structured, quantitative data.

The primary engineering challenge within this paradigm lies in enforcing deterministic behavior on an inherently probabilistic model. Left unconstrained, an evaluator LLM is prone to hallucination, semantic drift, and classification inconsistency. To ensure database stability and architectural reliability, two main engineering techniques must be applied:

1. **Structured Output Generation:** The model must be strictly bound to a deterministic schema (such as JSON Schema or Pydantic validation). This forces the LLM to output exclusively structured objects with rigid typing, enabling seamless integration with relational databases or data processing libraries like Pandas.

2. **Semantic Constraining through Negative Rules:** The prompt architecture must include strict boundaries to align the LLM's natural language understanding with the system's exact taxonomy. Without explicit negative constraints, an evaluator model frequently conflates the user's **meta-intent** (e.g., asking for a study plan) with the actual **knowledge domain** (e.g., Mathematics). Enforcing a rigid separation between structural actions and conceptual subjects is critical to maintaining a clean, non-redundant ontology over extended conversational lifecycles.

By establishing an LLM-as-a-Judge [3] as a background telemetry layer, a continuous evaluation loop is formed, providing the necessary data to dynamically update the multidimensional user profile at every turn of the conversation.

# Chapter 3

# Related Work

This chapter reviews the existing literature and state-of-the-art methodologies that intersect with the proposed architecture. The analysis focuses on recent advancements in prompt augmentation, context injection, and predictive student modeling, highlighting the theoretical foundations upon which this thesis builds and the specific limitations it aims to overcome.

## 3.1 K-LaMP: Knowledge-Augmented Language Model Prompting

In recent literature, the challenge of personalizing Large Language Models without incurring the computational overhead of continuous fine-tuning has gained significant attention. A prominent contribution to this domain is the Knowledge-Augmented Language Model Prompting (K-LaMP) framework, proposed by researchers at Microsoft [14].

K-LaMP addresses the inherent statelessness of LLMs specifically within the context of web search and query suggestion. The framework operates on the premise that a user's search experience can be drastically improved if the underlying generative model is aware of the user's historical objectives, interests, and prior knowledge. To achieve this, K-LaMP constructs a lightweight, entity-centric knowledge store for each individual user. Instead of retraining the model, the system analyzes the user's past search queries and browsing activities to extract relevant keywords and entities. When a new query is submitted, K-LaMP retrieves the historically related entities from this local store and dynamically injects them into the LLM's prompt. This zero-shot context augmentation effectively guides the model to generate personalized next-query suggestions that align with the user's implicit profile.

**Relation to the Proposed Architecture**

The K-LaMP framework provides robust empirical validation for the core architectural paradigm utilized in this thesis: that dynamic, user-specific context injection (Retrieval-Augmented Generation) is a highly efficient alternative to model fine-tuning for personalization tasks. Both K-LaMP and the architecture proposed in this research share the fundamental goal of intercepting the user's prompt and enriching it with a locally maintained user profile before passing it to the generative engine.

However, a critical comparative analysis reveals distinct differences in the depth and methodology of the user state representation. The K-LaMP architecture is fundamentally *extractive* and *flat*. It relies on the passive aggregation of public entities (e.g., matching search terms to a public knowledge graph) without evaluating the temporal or cognitive evolution of that knowledge. It knows *what* a user searched for, but it lacks the mathematical formalization to understand *how well* they retained it or whether their interest is currently waning.

In contrast, the Closed-Loop architecture proposed in this thesis advances the paradigm from flat entity matching to *active, multidimensional state tracking*. By implementing a background LLM-as-a-Judge [3] and an Exponential Moving Average (EMA) algorithm, the proposed system actively computes cognitive degradation (Lapse) and affective engagement (Interest). Furthermore, while K-LaMP injects unstructured lists of entities, the proposed architecture employs a Hierarchical Router to inject deterministic, multi-level ontologies, significantly reducing the prompt payload and preventing semantic drift. Thus, while K-LaMP successfully demonstrates the viability of prompt augmentation, this thesis extends the concept into a fully autonomous, memory-aware cognitive framework.

## 3.2 Exercise Recommendation Based on Ability Computation

A critical component of personalized educational technology is the ability to recommend content that accurately matches a user's evolving cognitive state. Recent literature has increasingly recognized that effective recommendation engines must transcend simple performance metrics to account for memory degradation and user engagement. A notable advancement in this area is the Learning and Forgetting Convolutional Knowledge Tracking Exercise Recommendation (LFCKT-ER) model proposed by Li and Niu [15].

The LFCKT-ER framework addresses the multifaceted nature of learning by computing not only a student's proficiency in specific knowledge concepts but also explicitly modeling their forgetting behavior over time. Furthermore, the system integrates non-cognitive dimensions, specifically filtering recommended exercises based on the student's learning stage preferences and matching the task difficulty to their computed expectations. This multidimensional approach aligns closely with the theoretical foundations of the dual-axis (cognitive and affective) tracking strategy implemented in this thesis.

**Relation to the Proposed Architecture**

The study by Li and Niu provides a robust empirical justification for the inclusion of the *Lapse* (memory decay) and *Interest* (preference) metrics within the proposed User Profile. It mathematically demonstrates that incorporating these variables drastically improves the relevance of system-generated recommendations compared to models that track knowledge in isolation. However, an analysis of the LFCKT-ER methodology highlights the strict limitations of traditional deep learning approaches when applied to modern, conversational AI interfaces. Because it relies on Convolutional Knowledge Tracking (CKT), the LFCKT-ER system is inextricably bound to highly formalized, closed-domain datasets. It requires a static, pre-existing database of exercises, manually tagged knowledge concepts, and deterministic input formats. It lacks the architectural flexibility to process unstructured natural language or dynamically adapt to Out-of-Domain (OOD) topics introduced spontaneously by the user.

The architecture proposed in this thesis explicitly overcomes this structural rigidity. By substituting traditional convolutional networks with the LLM-as-a-Judge paradigm [3], coupled with an Open Ontology strategy, the proposed system achieves the same advanced multidimensional profiling (Knowledge, Lapse, Interest) but extracts it entirely from free-form, zero-shot conversational text. Furthermore, instead of merely filtering a static database of predefined exercises, the generative LLM—guided by the Hierarchical Router—dynamically synthesizes tailored pedagogical interventions in real-time. This architectural pivot elevates the recommendation process from a rigid mathematical filtering task into a fluid, context-aware generative interaction.

## 3.3 Predicting Student Outcomes from EdTech Log Data

Beyond prompt augmentation and exercise recommendation, a fundamental challenge in educational technology is the extraction of actionable insights from continuous user interactions. A standard approach in Learning Analytics involves mining platform log data to infer user proficiency and predict future performance. A recent and highly relevant contribution to this domain is the study by Gao et al. [16], presented at the 15th International Learning Analytics and Knowledge Conference (LAK 25).

The research by Gao et al. investigates the viability of using short-term, granular EdTech log data (e.g., early-stage platform interactions, hint usage, and task completion times) as surrogate metrics to predict long-term student outcomes, such as end-of-year standardized test scores. By applying advanced predictive modeling to massive datasets of student telemetry, the authors demonstrate that continuous background tracking of micro-interactions contains highly predictive signals regarding a learner's cognitive trajectory and ultimate academic success.

**Relation to the Proposed Architecture**

The empirical findings of Gao et al. strongly validate the underlying premise of the tracking engine developed in this thesis: background telemetry is a powerful, non-intrusive method for modeling user state without interrupting the user experience. By continuously evaluating short-term interactions, a system can reliably estimate the broader cognitive profile of the user.

However, the methodology employed in traditional Learning Analytics heavily relies on structured, deterministic clickstream data. Models designed to parse clicks, completion times, or binary task outcomes are fundamentally incompatible with modern generative AI interfaces, where the primary mode of interaction is open-ended natural language. A conversational agent does not generate traditional clickstream logs; it generates unstructured semantic dialogues. Furthermore, the framework proposed by Gao et al. is inherently *predictive* and offline: it uses logs to forecast a future test score, usually powering analytics dashboards for educators rather than adapting the system's interface directly.

The Closed-Loop architecture presented in this thesis translates this predictive telemetry paradigm into a *generative, real-time* framework. Instead of mining structured clickstreams, the system deploys an LLM-as-a-Judge [3] to extract continuous quantitative metrics (Knowledge, Lapse, Interest) directly from unstructured conversational logs. Crucially, instead of using these metrics solely for offline outcome prediction, the architecture immediately feeds them back into the Hierarchical Router. This creates an autonomous, self-adjusting loop where short-term interaction data instantly dictates the semantic payload of the next generative prompt, transforming static log analysis into dynamic context-awareness.

## 3.4 Hierarchical State Compression in Industry: TencentDB Agent Memory

As AI agents transition from short-term conversational interfaces to long-horizon autonomous assistants, the inherent limitations of flat vector databases (standard RAG) become apparent. Flat log accumulation inevitably leads to context window bloat, escalating token consumption, and degraded retrieval precision. Addressing this industrial bottleneck, Tencent recently open-sourced the TencentDB Agent Memory framework [17], establishing a state-of-the-art baseline for local, hierarchical agent memory management.

The TencentDB architecture eschews flat conversational logging in favor of a dual-memory system optimized for token efficiency. It implements a four-tier progressive pipeline (semantic pyramid) for long-term memory: L0 (Raw Conversations), L1 (Atomic Facts), L2 (Scenario Blocks), and L3 (User Personas). Concurrently, it utilizes a symbolic short-term memory mechanism to compress verbose tool logs into lightweight structural representations. By forcing the generative agent to attend only to high-level abstractions (L3 Personas) rather than raw historical dialogue (L0), the framework reports a massive reduction in token consumption and a significant increase in task success rates.

**Relation to the Proposed Architecture**

The recent release of the TencentDB Agent Memory framework provides robust, cutting-edge industrial validation for the architectural choices made in this thesis. Both systems reject the "brute-force history accumulation" typical of naive RAG setups. Instead, they share the foundational principle that memory must be dynamically distilled and hierarchically routed to preserve context window integrity.

While the Tencent framework targets generalized agent task execution through its L0-L3 persona pyramid, the architecture proposed in this thesis specifically adapts this logic for the cognitive and educational domain. By replacing generic entity extraction with mathematical state-tracking equations (Knowledge, Lapse, Interest) computed via an asynchronous LLM-as-a-Judge [3], the proposed system translates the concept of "semantic compression" into a pedagogical tool. Furthermore, just as Tencent utilizes structural abstractions to drastically reduce token usage, the Hierarchical Router in this thesis actively minimizes the prompt payload, validating the hypothesis that structured state-tracking is the optimal path to computationally efficient personalization.

# Chapter 4

# Data Exploration and Processing

## 4.1 The MathE Dataset Overview

To empirically validate the Closed-Loop RAG architecture proposed in this thesis, real-world educational telemetry is required. The primary data source for this research is derived from the *MathE* platform [2], a digital ecosystem designed to support the teaching and learning of mathematics in higher education. The platform provides personalized assessments across 22 mathematical topics, supported by an extensive database. For the scope of this project, we focus on the MathE functionalities integrated into the *DataGEMS* project [1], which serves as the foundational bedrock for our continuous state tracking.

### 4.1.1 Database Schema and Relational Structure

The underlying relational database of the MathE ecosystem is highly complex, comprising multiple interconnected tables that handle users, educational offers, and ontological categorizations. The raw data dump provided for the exploratory phase of this research contained a wide array of tables and files. A brief description of the exported dataset structures is provided below:

- `Assessment_Information`: The critical telemetry log recording every evaluative interaction between the student and the platform over time. It is the only table containing the dynamic cognitive variables necessary for continuous state tracking.

- `Questions_Information`: Stores the core information regarding the mathematical assessments, including their textual wording, multiple-choice options (including the "I don't know" option), and their assigned algorithmic difficulty levels.

- `Teaching_Material_Information`: Contains metadata regarding the educational resources (e.g., video lessons, reviews, and written teaching materials) available on the platform, linking them to their respective creators and topics.

- `Keyword_Information`, `platform_materials-1` and `Keywords_by_Topic_Subtopic-1`: Store the ontological tags used to categorize questions and materials. These tables map the granular keywords to their parent `Topic` and `Subtopic`, acting as the semantic backbone of the platform.

- `PDF_Keywords`: An auxiliary junction table mapping the content of written documents (PDFs) to specific platform keywords.

- `MathE_public`: The public, anonymized subset of the MathE database utilized for general research purposes.

- `MathE_LLManswers`, `DataGEMS & MathE Example of questions and answers`, and `DataGEMS Chat examples`: Collections of sample interactions, prompts, and synthetic LLM-generated responses utilized for benchmarking the generative and reasoning capabilities of models within the DataGEMS project framework.

While tables such as `Teaching_Material_Information` and `Keyword_Information` define the static structural ontology of the educational offers, the fundamental requirement for dynamic user profiling resides strictly within the `Assessment_Information` table, which was therefore selected as the focal point of the subsequent Exploratory Data Analysis.

### 4.1.2 Exploratory Data Analysis (EDA) and Data Sparsity

An initial Exploratory Data Analysis (EDA) was conducted on the `Assessment_Information` dataset to assess data quality, sparsity, and variable distributions. To provide a rigorous structural overview of the primary data source, Table 4.1 details the dataset schema, including feature completeness and data types. This structural breakdown is critical for identifying which telemetry signals are robust enough to be included in the state-tracking mathematical formulation.

The extracted dataset consists of a total of 20,241 historical interaction logs.

Table 4.1: Dataset schema and feature completeness summary of the `Assessment_Information` table.

| # | Column | Non-Null Count | Dtype |
|---|---|---|---|
| 0 | student_id | 20,241 non-null | int64 |
| 1 | question_id | 20,241 non-null | int64 |
| 2 | answer | 20,241 non-null | int64 |
| 3 | date | 20,241 non-null | object |
| 4 | duration | 30 non-null | float64 |
| 5 | option_selected | 30 non-null | float64 |
| 6 | Topic | 20,241 non-null | object |
| 7 | Subtopic | 13,803 non-null | object |
| 8 | Lecturer_level | 20,241 non-null | int64 |
| 9 | Algorithm_level | 20,241 non-null | int64 |
| 10 | Typology | 20,241 non-null | object |

Key attributes identified for the computation of the user's cognitive state include:

- **`student_id` and `date`:** Essential for grouping interactions chronologically and computing the temporal memory decay (Lapse Score).
- **`Topic` and `Subtopic`:** The hierarchical semantic labels of the interaction.
- **`answer`:** A deterministic categorical variable indicating the user's specific response state:
  - *1* denotes a correct answer;
  - *0* denotes an incorrect answer;
  - *-1* indicates that the user explicitly selected the "I don't know" option.
- **`Algorithm_level`:** A discrete metric ranging from 1 to 6, dynamically assigned by the platform's algorithm to represent the intrinsic difficulty of the assigned question.

A critical finding from the EDA is the extreme sparsity of behavioral metrics. Specifically, the `duration` (time taken to complete an assessment) and `option_selected` columns contain only 30 non-null values out of the

20,241 records. Consequently, these variables had to be completely discarded from the mathematical formulation of the user profile. The absence of reliable completion-time telemetry strictly limits the cognitive tracking engine to accuracy-based metrics (`answer`) weighted by task difficulty (`Algorithm_level`).

Similarly, metadata columns such as `question_id`, `Lecturer_level`, and `Typology` were actively excluded from the state-tracking equations. While `question_id` serves purely as an internal relational key, `Typology` designates the user's platform role (e.g., Student, Admin, Lecturer, or Lecturer Reviewer) rather than providing cognitive or behavioral telemetry. Furthermore, `Lecturer_level` represents a static difficulty estimation assigned by a human teacher. This was deliberately superseded by the dynamically computed `Algorithm_level` to ensure a more objective and standardized weighting of task complexity across the platform.

Furthermore, the `Subtopic` column exhibits a $\sim$31% null rate (only 13,803 non-null entries). To address this semantic incompleteness without discarding valuable telemetry data, a hierarchical data imputation strategy was applied during preprocessing. Specifically, missing subtopic values were automatically populated using their corresponding parent `Topic` label (Hierarchical Fallback). This approach preserves the integrity of the temporal and performance metrics (allowing the continuous calculation of the Knowledge Score for those interactions) while ensuring that the semantic routing mechanisms (detailed in Chapter 5) possess a deterministic text string to map against, effectively preventing null-reference errors during context extraction.

To visually substantiate these findings, Figure 4.1 displays the distributions of the performance and difficulty metrics across the dataset. The distinct presence of the $-1$ state in the `answer` distribution highlights moments of explicit uncertainty, while the right-skewed variance in `Algorithm_level` confirms the variance in task difficulty.

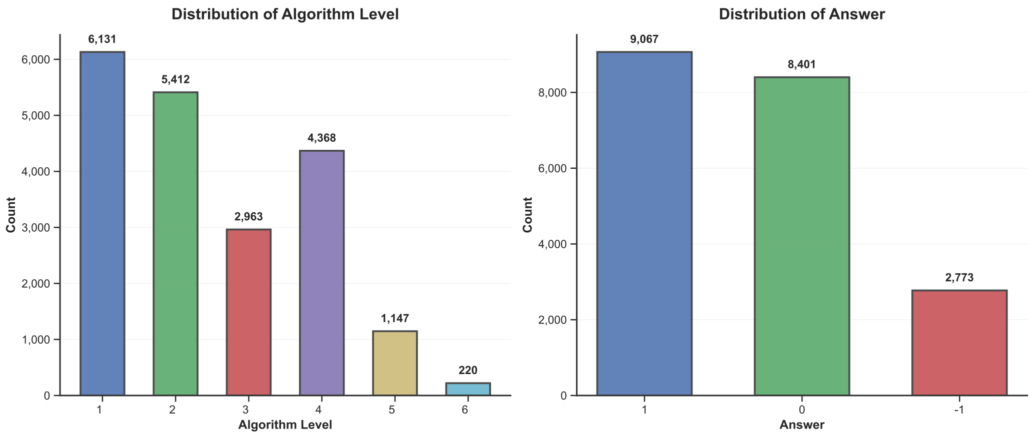


Figure 4.1: Distributions of user responses (`answer`) and algorithmic difficulty (`Algorithm_level`) in the MathE dataset.

## 4.2 Data Preprocessing and Metric Formulation

To transition from raw, static assessment logs to a dynamic, multidimensional user profile, the variables identified in the EDA phase must be mathematically combined. The objective of this pipeline is to map sporadic telemetry events into continuous, normalized metrics representing the user's Cognitive State (Knowledge and Memory Lapse) and Affective State (Interest).

### 4.2.1 Cognitive and Affective Metric Foundation

The base Knowledge Score ($K_{event}$) for a single assessment event is not a flat binary outcome, but a weighted metric accounting for the intrinsic difficulty of the task. Using the variables extracted from the `Assessment_Information` table, the system maps the raw categorical response ($a \in \{-1, 0, 1\}$) into a binary multiplier $m \in \{0, 1\}$.

In this implementation, both 0 (an incorrect guess) and $-1$ (an explicit "I don't know") are mapped to $m = 0$. From a strict knowledge-tracing perspective, both states reflect a lack of current cognitive mastery and must yield zero points for that specific interaction. While the explicit admission of ignorance ($-1$) is pedagogically valuable for preventing false positives (e.g., lucky guesses), it mathematically equates to an incorrect answer when calculating the raw event score. The event knowledge is then formulated as a function of this multiplier $m$ scaled by the question's difficulty level ($d \in \{1, \ldots, 6\}$):

$$K_{event} = (m \cdot d) \cdot W_k \tag{4.1}$$

Where $W_k$ represents a normalization constant utilized to scale the raw score to a standard $0-100$ index. Because the algorithm's difficulty level ($d$) reaches a maximum value of 6, $W_k$ is defined by dividing the target scale (100) by the maximum possible cardinality of the event ($6 \cdot 1$), resulting in $W_k = \frac{100}{6} \approx 16.67$. This ensures that correct mastery of the most complex concept ($d = 6$) yields a perfect score of 100, while a correct answer on a trivial task ($d = 1$) yields a proportionally lower score, and failure ($m = 0$) results in zero. Similarly, the Affective State is structured as a continuous *Interest Score* ($I \in [0, 100]$). In the Closed-Loop architecture, this metric is extracted by the asynchronous LLM Evaluator based on semantic conversational cues, quantifying the learner's engagement without requiring explicit questionnaires.

#### From Event-Level Logs to Macro-Topic Aggregation

Because the raw `Assessment_Information` table logs telemetry at the extreme micro-level (individual question interactions), the data must be structurally aggregated to be meaningful for an LLM routing engine. Tracking mastery at the level of a single quiz ID is computationally inefficient and pedagogically irrelevant for conversational generation.

Therefore, during the preprocessing phase, after computing the isolated $K_{event}$ and $L$ scores for every single question answered by a specific `student_id`, the dataset was grouped by its ontological labels (`Topic` and `Subtopic`). The baseline cognitive profile for each Subtopic was established by calculating the arithmetic mean of all corresponding $K_{event}$ and $L$ scores logged up to that point. This aggregation transforms a fragmented history of individual quiz attempts into a unified, macro-level *State-Tracking Vector*, which serves as the foundational starting point for the continuous EMA updating described in the subsequent sections.

### 4.2.2 Asynchronous State Updating via Exponential Moving Average (EMA)

A core requirement of this architecture is to maintain a persistent profile resilient to statistical noise. Single interactions (e.g., a momentary lapse in concentration) should not drastically overwrite the user's longitudinal history. To ensure stability, an Exponential Moving Average (EMA) algorithm is applied to both $K$ and $I$ after every valid interaction event:

$$S_t = \alpha \cdot S_{t-1} + (1 - \alpha) \cdot E_t \tag{4.2}$$

Where $S_t$ denotes the smoothed state at step $t$, $E_t$ is the event score derived from the current interaction, and $\alpha \in [0, 1]$ is the smoothing factor. The specific value of $\alpha = 0.8$ was determined experimentally. Empirical testing revealed that lower values caused the profile to fluctuate too aggressively in response to single anomalies (e.g., a distracted wrong answer drastically dropping the overall score), whereas values above 0.9 made the profile excessively rigid and unresponsive to genuine learning progress. Setting $\alpha = 0.8$ optimally balances responsiveness with historical stability, heavily weighting historical mastery (80%) to ensure that the system's pedagogical recommendations do not pivot erratically.

### 4.2.3 Modeling Temporal Decay: The EdTech Forgetting Curve

While EMA tracks mastery accumulation, human memory is inherently subject to temporal degradation. To quantify this, the architecture computes a dynamic *Lapse Score* ($L$) using an EdTech-optimized adaptation of the Ebbinghaus Forgetting Curve [11].

The Lapse Score is computed as a continuous function of the time elapsed ($\Delta t$ in days) since the last interaction with a subtopic:

$$L(\Delta t) = 100 \cdot e^{-\frac{\max(\Delta t, 0)}{S}} \tag{4.3}$$

The *Strength of Memory* ($S$) is not static; it is modeled as a custom function of the user's historical Knowledge ($K$):

$$S = 30 + \frac{K}{2} \tag{4.4}$$

The parameters of this equation were defined experimentally to approximate the temporal dynamics of a typical university semester. The base constant of 30 establishes a minimum memory strength (in days) even for concepts with near-zero knowledge, preventing the exponential decay from instantly zeroing out the profile after a few days of inactivity. The $\frac{K}{2}$ component dynamically extends this strength up to a maximum of 80 days ($30 + \frac{100}{2}$) for fully mastered topics. While a rigorous psychological calibration of these cognitive constants is beyond the scope of this computer science proof-of-concept, this mathematical formulation effectively models the reality that robust mastery acts as a buffer against rapid forgetting.

### Asymmetric Retrieval Strength Accumulation

To prevent the student profile from overestimating long-term retention based on a single successful interaction, the architecture implements an asymmetric recovery function for the Retrieval Strength. Drawing upon the *New Theory of Disuse* [18]–which separates structural Knowledge (Storage Strength) from immediate memory access (Retrieval Strength)—the system avoids resetting the *Lapse Score* directly to its maximum value (100 = *"Not Lapsed"*) after a learning event. Instead, it applies a weighted linear interpolation between the residual accessibility score ($L_{current}$) and the theoretical maximum:

$$L_{new} = (L_{current} \cdot \alpha_{lapse}) + (100.0 \cdot (1 - \alpha_{lapse})) \tag{4.5}$$

Where $\alpha_{lapse} \in [0, 1]$ represents the retrieval inertia factor. By configuring this parameter to a high value ($\alpha_{lapse} = 0.85$), the system deliberately imposes a strict recovery curve. This mathematical friction is fundamentally justified by the tracking granularity of the *State-Tracking Vector*. Because the architecture operates at the *Subtopic* level—encompassing broad knowledge domains rather than atomic concepts—a single successful interaction provides insufficient evidence to validate the recovery of the entire macro-area. Consequently, a severely lapsed Subtopic requires a sustained sequence of active learning events to fully restore the accessibility state to an unlapsed condition.

## 4.3 Synthetic Data-Set Generation and Semantic Discretization

To stress-test the architectural limits of the Hierarchical Router and the telemetry precision of the LLM Evaluator, the original MathE dataset was augmented with a synthetic repository. This process served two primary purposes: expanding the topical breadth beyond pure mathematics to validate cross-domain adaptability and ensuring a sufficient volume of interaction logs to observe the convergence of the Exponential Moving Average (EMA) algorithm.

Through continuous execution, the system has successfully generated and maintained a longitudinal *Context Data* repository exceeding 1,000 unique interaction logs. This high-volume dataset allows the system to maintain a granular, multi-topic cognitive profile for the student, demonstrating high-fidelity persistence over time.

### 4.3.1 Semantic Discretization Methodology

Raw quantitative metrics $(0 - 100)$ are not natively expressive for a generative LLM's prompt window. Passing raw floats to the model increases the risk of "numerical hallucination," where the model struggles to interpret the semantic significance of a score like 73.4 versus 74.1. To bridge this gap, a *Semantic Discretization* pipeline was implemented, mapping continuous scores into discrete, human-readable labels that reflect pedagogical states.

This mapping process utilizes a custom binning methodology defined by pedagogical expertise:

- **Knowledge Labels (K-Cat):** Ranges were discretized into five tiers: *Extremely low* (0–20), *Low* (20–40), *Moderate* (40–60), *High* (60–80), and *Extremely high* $(80 - 100)$.
- **Lapse Labels (L-Cat):** Discretized into: *Extremely lapsed* $(0 - 20)$, *Highly lapsed* $(20 - 40)$, *Moderately lapsed* $(40 - 60)$, *Slightly lapsed* $(60 - 80)$, and *Not lapsed* $(80 - 100)$.
- **Interest Labels (I-Cat):** Mapped into five tiers: *Extremely low* $(0 - 20)$, *Low* $(20 - 40)$, *Moderate* $(40 - 60)$, *High* $(60 - 80)$, and *Extremely high* $(80 - 100)$.

This methodology significantly reduces the variance in LLM output, ensuring consistent pedagogical behavior across different domains. The resulting structured profile, referred to as the *State-Tracking Vector*, consists exclusively of the relevant `Topic`, `Subtopic`, and the discretized categorical labels (`K-Cat`, `L-Cat`, `I-Cat`). By injecting this distilled semantic vector directly into the system prompt, the tutor can modulate its tone, complexity, and proactivity with high precision, avoiding the noise associated with raw numerical inputs. Table 4.2 provides a representative snapshot of the resulting `context_data.csv`, showing both the underlying continuous scores and their corresponding semantic discretizations.

Table 4.2: Sample snapshot of the longitudinal cognitive profile with discretized categories.

| **Topic** | **Subtopic** | **K** | **K-Cat** | **L** | **L-Cat** | **I** | **I-Cat** |
|---|---|---|---|---|---|---|---|
| Derivation | II Derivatives | 17.0 | Ex. Low | 32.0 | Ex. Lapsed | 19.0 | Ex. Low |
| Integration | Triple Int. | 40.0 | Low | 2.0 | Ex. Lapsed | 13.0 | Ex. Low |
| Probability | Comb. Cal. | 22.0 | Low | 91.0 | Not Lapsed | 43 | Moderate |
| ... | ... | ... | ... | ... | ... | ... | ... |

# Chapter 5

# Architectural Design and Implementation

Before analyzing the architectural evolution of the system, it is necessary to specify the underlying technological stack driving the pedagogical agent. To optimize the trade-off between computational latency, operational cost, and reasoning capability, the architecture was designed as a multi-agent system utilizing specialized models from the Gemini 2.5 family [19]. Specifically, the generative Pedagogical Tutor is powered by `gemini-2.5-flash` to ensure robust conversational and didactic capabilities, while all intermediate data-processing layers—namely the State-Tracking Router and the asynchronous Evaluator—are powered by `gemini-2.5-flash-lite`, prioritizing high-speed JSON extraction and minimal API footprint.

## 5.1 The Baseline: Naive RAG Approach

The initial architectural iteration of the tracking system followed a *Naive RAG (Retrieval-Augmented Generation)* pattern. In this baseline configuration, the system attempted to maintain conversational and cognitive context by retrieving the entire student profile from the persistent `context_data.csv` repository and injecting it directly into the LLM's system prompt prior to every user query. While conceptually simple and easy to implement, this brute-force approach rapidly proved to be mathematically and operationally unsustainable for a continuous educational application.

Quantitative benchmarking of the Naive approach highlighted a severe correlation between longitudinal database growth and system degradation. An analysis of the system telemetry (`generator_usage.csv`) revealed that injecting the full dataset at its current volume (+1000 rows x 5 columns) resulted in an unsustainable prompt payload, averaging 26,877 input tokens per interaction. This massive data injection induced three critical points of architectural failure:

1. **Context Window Exhaustion and Cost Inefficiency:** Passing nearly 27,000 tokens per turn severely limits the length of the actual conversational history that can be maintained. Furthermore, in a production environment utilizing proprietary APIs, this static payload results in an exponential and unnecessary inflation of computational costs, which will scale linearly as the student's dataset grows over time.

2. **The Latency vs. Global Reasoning Trade-off:** Telemetry data indicated that the average generation latency for the Naive model exceeded 13 seconds per turn. This high latency is a direct consequence of the LLM physically processing and cross-referencing the entirety of the student's database before generating an answer. While this "brute-force" reading grants the Naive model a significant advantage—the ability to successfully answer holistic, global profile queries (e.g., "What is my favorite hobby?" or "What am I best at?") by scanning all available rows—the computational cost is too high.

3. **Semantic Noise (The "Lost in the Middle" Phenomenon [20]):** Beyond hardware and cost limitations, the Naive approach degraded the LLM's actual reasoning capabilities. By providing an overwhelming amount of irrelevant information (e.g., historical scores on *Analytic Geometry* while the user was exclusively asking about *Fluid Dynamics*), the model suffered from the "Lost in the Middle" phenomenon [20]. The LLM struggled to isolate the active cognitive state, occasionally hallucinating recommendations based on adjacent, irrelevant database rows.

Consequently, it became evident that a "brute-force" RAG architecture was fundamentally incompatible with the goal of continuous, scalable learning tracking. This baseline provided the empirical evidence necessary to hypothesize that a *Hierarchical Closed-Loop* architecture—capable of routing requests and filtering state vectors—was a strict prerequisite for the system's viability.

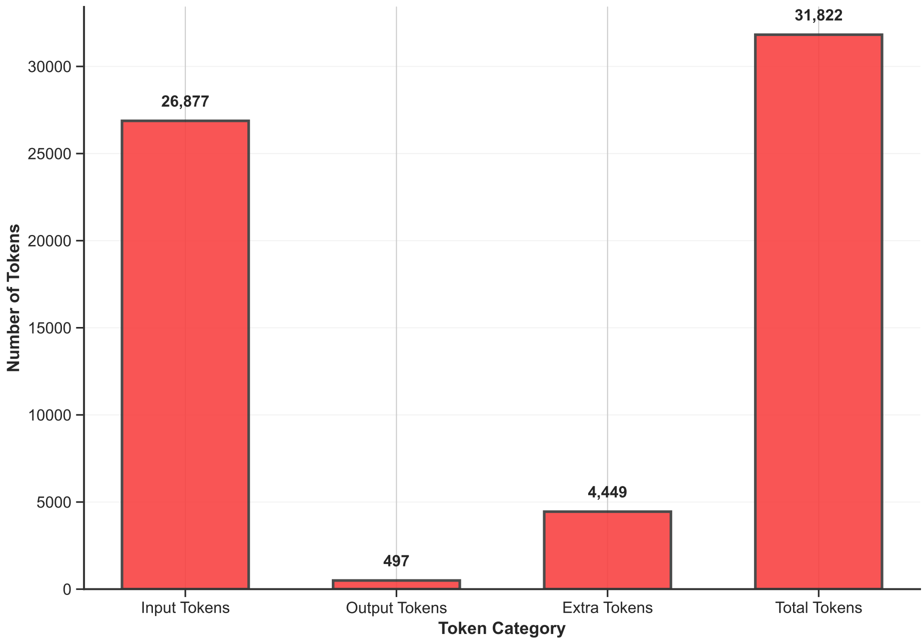


Figure 5.1: Token usage breakdown for the Generator LLM under the Naive RAG baseline. The chart highlights the severe disproportion between the massive input payload (26,877 tokens) and the generative output (497 tokens), visually demonstrating the context window exhaustion caused by injecting the unrouted database.

## 5.2 Intelligent State-Tracking Router

To overcome the payload limitations of the Naive approach, an intermediate semantic extraction layer was engineered: the *State-Tracking Router*. Instead of injecting the entire database, the Router acts as an autonomous triage agent. It intercepts the user's raw query, analyzes its semantic intent, and extracts only the specific topics relevant to the current conversation. The generative LLM is then provided exclusively with the filtered slice of the database, drastically reducing the prompt payload.

### 5.2.1 Overcoming Semantic Ambiguity: The FRB vs. HRB Trade-off

The first iteration of this extraction layer utilized a *Flat Routing* (FRB) methodology. The Router was provided with a simple, linear list of all available macro-topics in the database and asked to output the closest match. While this approach was highly token-efficient at the routing stage—consuming an average of only 748 input tokens and resolving in 0.99 seconds—it introduced a severe semantic vulnerability: probabilistic guessing.

Because the Flat Router only had visibility over broad topics (e.g., "Mathematics", "Physics", "History"), it frequently struggled with Semantic Ambiguity when faced with granular user queries. If a user asked a highly specific question such as *"Can you help me with Triple Integrals?"* or *"I have an exam on Fluid Dynamics,"* the FRB was forced to probabilistically guess which parent topic these concepts belonged to. Without a mapping of the underlying subtopics, this guesswork occasionally led to retrieval misses, leaving the Tutor LLM completely blind to the user's actual profile.

To eliminate this guesswork, an alternative architecture was developed: *Hierarchical Context Routing* (HRB). In this configuration, the Router is no longer fed a flat list. Instead, it is dynamically injected with a deterministic parent-child relationship map (e.g., `Topic` $\rightarrow$ `Subtopics`).

By providing this complete topological map, the HRB model achieves higher extraction accuracy. If the user mentions "Fluid Dynamics," the Router deterministically traverses the map to extract the correct parent topic ("Physics"), successfully pulling all relevant sub-nodes into the context window of the Generator LLM.

This structural precision is empirically reflected in the volume of retrieved context. Telemetry data indicates that the FRB, compensating for its semantic uncertainty, tends to over-extract, pulling an average of 2 macro-topics and 15 subtopics per query in an attempt to capture the correct domain. In contrast, the HRB's deterministic traversal allows for a surgical retrieval, narrowing the context to an average of just 1 strictly relevant macro-topic and 9 associated subtopics. This targeted extraction eliminates the "shotgun approach" of the FRB, significantly reducing the semantic noise injected into the downstream generation phase.

However, this architectural alternative cannot be classified as strictly superior, as it introduces a significant operational trade-off. Injecting the full hierarchical database map forces a massive expansion of the prompt payload at the routing stage. Telemetry data reveals that the HRB Router consumes 9,427 input tokens per turn, a drastic 12-fold increase compared to the FRB baseline. Interestingly, despite this massive payload expansion, the latency penalty is marginal, rising only from 0.99 to 1.02 seconds.

Therefore, the architectural decision between FRB and HRB represents a classic engineering compromise strictly regarding API costs rather than execution speed: FRB optimizes for computational cost-efficiency at the risk of semantic misses and noise over-extraction, while HRB deliberately sacrifices token economy at the routing layer to guarantee flawless, targeted context retrieval.

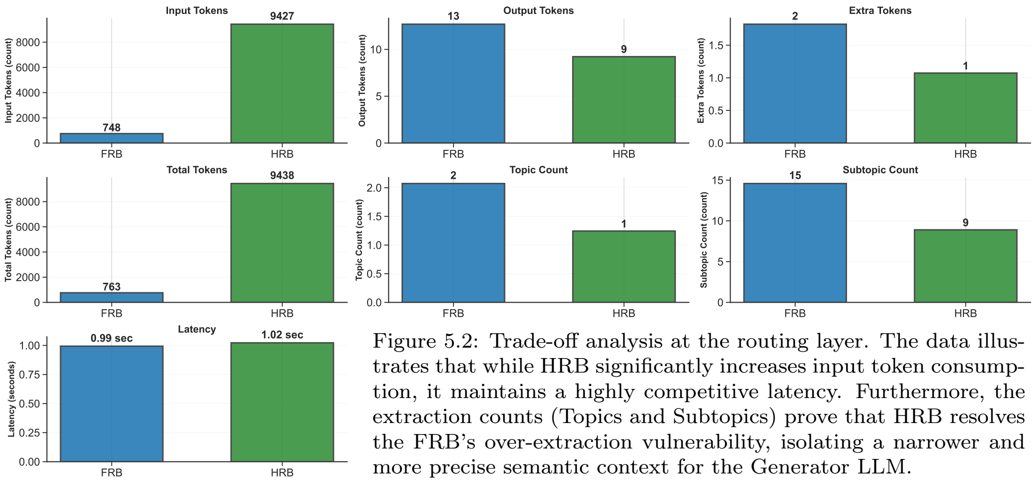


Figure 5.2: Trade-off analysis at the routing layer. The data illustrates that while HRB significantly increases input token consumption, it maintains a highly competitive latency. Furthermore, the extraction counts (Topics and Subtopics) prove that HRB resolves the FRB's over-extraction vulnerability, isolating a narrower and more precise semantic context for the Generator LLM.

### 5.2.2 Architectural Limitations: The Global Profile Query Dilemma

While the HRB significantly optimizes localized pedagogical queries, isolating the context introduces a known structural limitation regarding *Global Profile Queries*. Because the Router is strictly designed to perform "Local Routing" (matching a specific domain query to specific database rows), it mathematically fails when a user asks holistic meta-questions, such as: *"What is the subject I am worst at?"* or *"Based on my entire history, what should I study today?"*.

In these scenarios, the HRB attempts to find a specific academic topic matching the query, fails, and consequently passes an empty context window to the LLM. While the Naive model naturally excels at these queries by having access to the entire CSV, injecting thousands of tokens solely to support occasional meta-queries negates the entire purpose of the routing architecture.

In enterprise-scale deployments, this dilemma is resolved by implementing an upstream *Intent Classifier* prior to the HRB layer. This classifier acts as a semantic switch: if it detects a specific domain query (e.g., "Explain integrals"), it triggers the standard token-efficient HRB.

Conversely, if it detects a holistic meta-query, it bypasses the semantic router and triggers a *Reverse Metric-Driven Routing* protocol. Instead of searching for text-based topic matches, this mechanism performs a programmatic filter directly on the database metadata (Knowledge, Interest, and Lapse scores). For instance, if the student asks, "What should I review today?", the system filters the database for rows containing a "Highly Lapsed" value, extracts only the associated topic names, and injects this minimal array into the LLM's prompt.

By inverting the routing logic from "Topic-to-Metric" to "Metric-to-Topic," the system can seamlessly answer global performance questions while maintaining strict token efficiency and offloading the computational burden from the LLM to a deterministic database filter.

## 5.3 Proactive Recommendation Strategy

A fundamental objective of the DataGEMS educational integration is to transition the LLM from a passive answering engine into a proactive pedagogical agent. In a standard conversational setup, an LLM typically waits for the user's next command. However, effective tutoring requires guiding the student's learning trajectory, suggesting next steps based on their demonstrated strengths and weaknesses.

To achieve this, a *Proactive Recommendation Matrix* was strictly embedded within the LLM's system prompt. Instead of allowing the model to hallucinate study paths, the prompt explicitly binds the LLM's generative behavior to the *State-Tracking Vector* extracted by the Router.

The pedagogical behavior is modulated across two primary dimensions:

1. **Cognitive Modulation (Knowledge and Lapse):** The prompt dictates specific instructional strategies based on the discretized categories. For instances of *'Extremely low' / 'Low knowledge'*, the Tutor is constrained to suggest foundational basics. Conversely, for *'Extremely high knowledge'*, it is directed to propose advanced problems. Crucially, the model utilizes the Lapse category to differentiate its approach: a profile showing *'Moderate knowledge'* but *'Highly lapsed'* triggers a quick memory refresher, whereas a *'Not lapsed'* profile triggers direct practice execution.

2. **Affective Modulation (Interest Score):** To prevent user disengagement, the Tutor integrates the user's Interest category into its generative strategy. When addressing topics with *'Extremely high interest'*, the LLM is instructed to leverage engaging, real-world applications. If the interest is *'Low'*, the system attempts to pivot the explanation toward interdisciplinary connections designed to spark curiosity.

To ensure full experimental reproducibility and to transparently document the prompt engineering process, the exact System Prompt instructions governing the Tutor's proactive behavior are reported below:

**System Prompt Excerpt: Proactive Recommendation Matrix**

```
1. RECOMMENDATION MATRIX:
   - 'Extremely low' / 'Low knowledge': Suggest foundational
     basics.
   - 'Extremely lapsed' / 'Highly lapsed' (with moderate
     knowledge): Suggest quick memory refreshers.
   - 'High' / 'Extremely high knowledge': Suggest advanced
     problems/applications.
   - 'Not lapsed' / 'Slightly lapsed' (with low knowledge): Focus
      on practice.

2. If the user asks for advice on how to improve, ALWAYS provide
   2-3 specific, actionable suggestions based on their scores.

3. INTEREST SCORE:
   - If the user has a 'High interest' score, suggest engaging,
     real-world applications. If 'Low interest', suggest ways to
      spark curiosity.
   - If the user asks for suggestions, keep in mind to provide
     suggestions that are in line with their interest level.
   - If you have to use general knowledge due to lack of data,
     use the interest score to guide your suggestions.
```

## 5.4 The Closed-Loop RAG Architecture

The defining engineering contribution of this system is the implementation of a *Closed-Loop* tracking cycle. Unlike traditional RAG frameworks that merely read from a static database, this architecture actively updates the persistent `context_data.csv` repository after every conversational turn, ensuring the user profile continuously reflects the latest cognitive state.

This loop is powered by an asynchronous *LLM-as-a-Judge [3]* (the Evaluator). Immediately after the LLM (`gemini-2.5-flash`) generates a response to the user, a secondary lightweight LLM (`gemini-2.5-flash-lite`) is triggered in the background. It analyzes the specific user-tutor interaction and extracts the numerical event scores ($K_{event}$ and $I_{event}$), which are then processed by the Exponential Moving Average (EMA) and Asymmetric Retrieval functions described in Chapter 4.

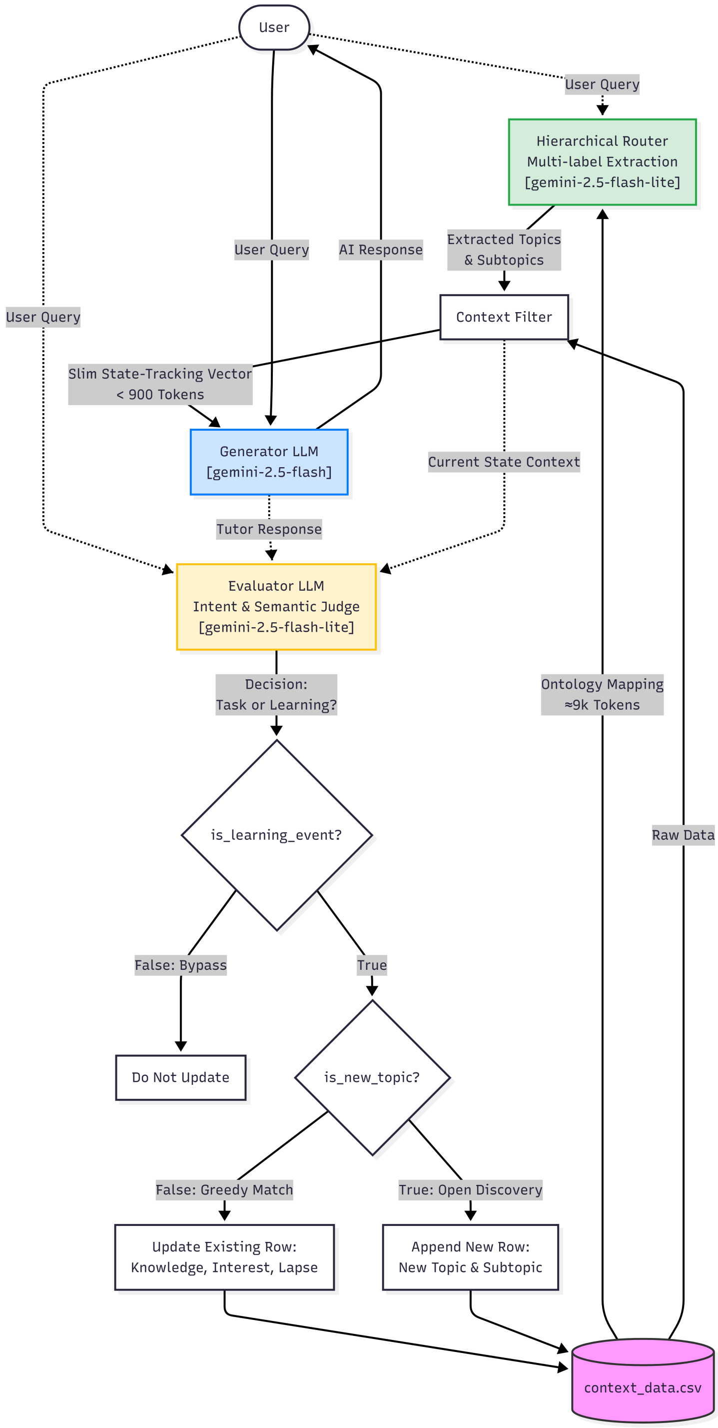


Figure 5.3: The definitive Closed-Loop Hierarchical Router-Based (HRB) Architecture. The system dynamically filters the context via the Router and continuously updates the user profile through the asynchronous Evaluator LLM.

### 5.4.1 Data Integrity via Semantic Bypass (The "Kill Switch")

A critical vulnerability in continuous conversational tracking is the pollution of the cognitive database with non-pedagogical data. In a free-form chat interface, a student will not only discuss academic concepts but will inevitably issue operational or meta-cognitive commands (e.g., *"Generate a 7-day study plan"*, *"What is my current status in History?"*, or *"Give me a list of YouTube channels"*).

If the system processes these operational tasks as learning events, it would incorrectly update the user's mastery and memory lapse scores based on administrative queries, fatally corrupting the profile's pedagogical validity. To protect database integrity, a *Semantic Bypass* (or "Kill Switch") was engineered into the Evaluator's prompt.

Before evaluating any numerical score, the Judge must output a boolean flag: `is_valid_tracking_event`. The prompt strictly instructs the LLM to classify the interaction into one of two categories:

- **Learning/Exploration Event:** Discussing concepts, theories, facts, or solving problems (Flag = `True`).
- **Task/Execution Event:** Requesting plans, summaries, status checks, or resources (Flag = `False`).

If the flag evaluates to `False`, the system triggers an early return sequence, entirely bypassing the EMA and Lapse updates. This architectural failsafe guarantees that the *State-Tracking Vector* exclusively reflects genuine cognitive mastery, isolated from standard conversational noise.

### 5.4.2 From Closed to Open Ontology via Conditional Prompting

Another major challenge in designing the asynchronous Evaluator was managing the taxonomy of the student's knowledge database. Initially, to prevent the system from hallucinating keys and duplicating existing rows, the Evaluator was constrained to a *Closed Ontology.* It was forced through strict prompt engineering to only extract and copy-paste exact string matches from the retrieved context. While this ensured database stability, it introduced a severe limitation: the system became entirely incapable of tracking Out-of-Domain (OOD) topics spontaneously introduced by the user (e.g., shifting the conversation to a known topic like the history of the Roman Empire when the user's request pertained to Medieval History, a topic not present in the ontology).

To overcome this constraint without sacrificing database integrity, a *Conditional Output Routing* mechanism was engineered within the LLM-as-a-Judge [3] JSON schema. By introducing a secondary boolean flag (`is_new_topic`), the model is forced to perform an intermediate logical reasoning step before text generation:

1. **Exact Match Strategy:** If the user's intent maps to an existing topic in the injected context, the model sets the boolean flag to `false` and strictly reuses the established ontology strings.

2. **Open Discovery Strategy:** If the interaction explores an untracked domain, the model sets the flag to `true`. This unlocks a secondary instruction set, authorizing the Evaluator to dynamically generate a new `Topic` and `Subtopic` to be appended to the database.

To enforce these logical pathways, the background Evaluator was strictly constrained to output its analysis using the following structured JSON schema:

**Evaluator JSON Schema Excerpt: The Semantic Bypass**

```
{

"is_valid_tracking_event": {"type": "BOOLEAN"},
"is_new_topic": {"type": "BOOLEAN"},
"topic": {"type": "STRING"},
"subtopic": {"type": "STRING"},
"interaction_knowledge": {"type": "NUMBER"},
"interaction_interest": {"type": "NUMBER"}

}
```

### 5.4.3 Enforcing Domain-Knowledge Semantics over Meta-Intents

During the testing phase of the Open Discovery Strategy, an anomaly emerged regarding Natural Language Understanding (NLU) alignment, known as *Semantic Bleed*.

Firstly, when a user asked for help preparing for an upcoming test on a new subject, the Evaluator frequently classified the new subtopic as *"Test Preparation Strategies"* rather than the actual academic discipline. The LLM conflated the *user intent* with the *knowledge domain*. Because educational Knowledge Tracing requires tracking cognitive mastery over specific subjects rather than meta-learning activities, a strict semantic constraint was introduced. The prompt explicitly forbids the generation of meta-activity labels (e.g., "Homework Help") and enforces the extraction of pure academic classifications.

Secondly, the Evaluator exhibited a strong bias toward the retrieved context, lazily mapping new Out-of-Domain interactions (e.g., 'Medieval History') to existing, loosely related topics in the database (e.g., 'The Roman Empire') simply because both belonged to the broad 'History' category. To mitigate this Generalization Trap, strict negative constraints and one-shot examples were introduced into the Evaluator's prompt, enforcing a zero-tolerance policy for loose matching and ensuring strict separation between broad categories and specific subtopics. This ensured that the dynamically generated ontology remained educationally relevant and structurally coherent with the rest of the MathE dataset.

To maintain database integrity while allowing for the Open Discovery Strategy, the Evaluator is constrained by a "Greedy Matching" protocol. This prevents the unnecessary generation of new ontology paths when the user's intent can be safely mapped to existing domains:

**System Prompt Excerpt: Evaluator Ontology Rules**

```
CRITICAL INSTRUCTIONS FOR TOPIC SELECTION:
- Step 1 (GREEDY MATCHING): Review the CURRENT STATE. Can the
    core subject of this interaction be reasonably grouped under
    an existing Subtopic? If yes, reuse it.
- Step 2: If a match is found, set "is_new_topic" to false.
    EXACTLY COPY-PASTE the "Topic" and "Subtopic" strings.
- Step 3: Set "is_new_topic" to true ONLY IF the interaction
    represents a complete paradigm shift to a drastically
    different academic subject.
- Step 4: If creating a new topic, it MUST be a pure knowledge
    domain.
```

# Chapter 6

# Evaluation and Results

The objective of this chapter is to empirically validate the proposed *Hierarchical Router-Based* (HRB) architecture. The evaluation is divided into two dimensions: a quantitative analysis of system telemetry to assess computational efficiency, and a qualitative review of interaction logs to demonstrate the pedagogical effectiveness of the autonomous tracking loop.

## 6.1 Quantitative Performance Analysis

To objectively measure the architectural improvements, system telemetry was aggregated across the three developed configurations: the Naive RAG baseline, the Flat Router-Based (FRB) model, and the definitive Hierarchical Router-Based (HRB) model. The primary evaluation metrics are token consumption (acting as a proxy for API operational costs) and inference latency (acting as a proxy for user experience fluidness).

### 6.1.1 Generator-Level Context Compression

The most immediate impact of the semantic routing layer is observed at the core text-generation stage. As visualized in Figure 6.1, the Naive baseline forced the Generator LLM to process the unrouted persistent database, resulting in an unsustainable prompt payload averaging 26,877 input tokens.

The implementation of semantic extraction successfully dismantled this computational bottleneck, though the two routing methodologies exhibited distinct behaviors. The Flat Routing (FRB) approach compressed the generator input payload to an average of 944 tokens. However, the Hierarchical Routing (HRB) achieved the most aggressive context compression, lowering the generator input to just 866 tokens. This discrepancy mathematically corroborates the hypothesis presented in Section 5.2.1: because the FRB relies on probabilistic guessing, it tends to over-extract contextual subtopics, injecting semantic noise into the Generator. The HRB's deterministic mapping allows for a surgical extraction, delivering a leaner, highly targeted state vector.

Crucially, the telemetry data in Figure 6.1 exposes a hidden multiplier effect in operational costs, represented by the *Extra Tokens* metric (calculated as *Total Tokens - (Input Tokens + Output Tokens)*). These extra tokens are generated when the LLM dynamically triggers external background tools, such as internet searches for real-time grounding. During tool execution, the model enters a multi-turn loop, forcing the API to iteratively re-process the entire conversational context. In the Naive configuration, a single web search triggers a massive, redundant re-injection of the unrouted database, resulting in an average overhead of 4,449 extra tokens per turn. Conversely, the HRB architecture mitigates this cumulative bloat, limiting the overhead to just 502 extra tokens. This provides empirical evidence that context window exhaustion does not scale linearly; when tool-use is integrated into a system, failing to filter the context creates a multiplicative compounding effect that drastically inflates the API payload.

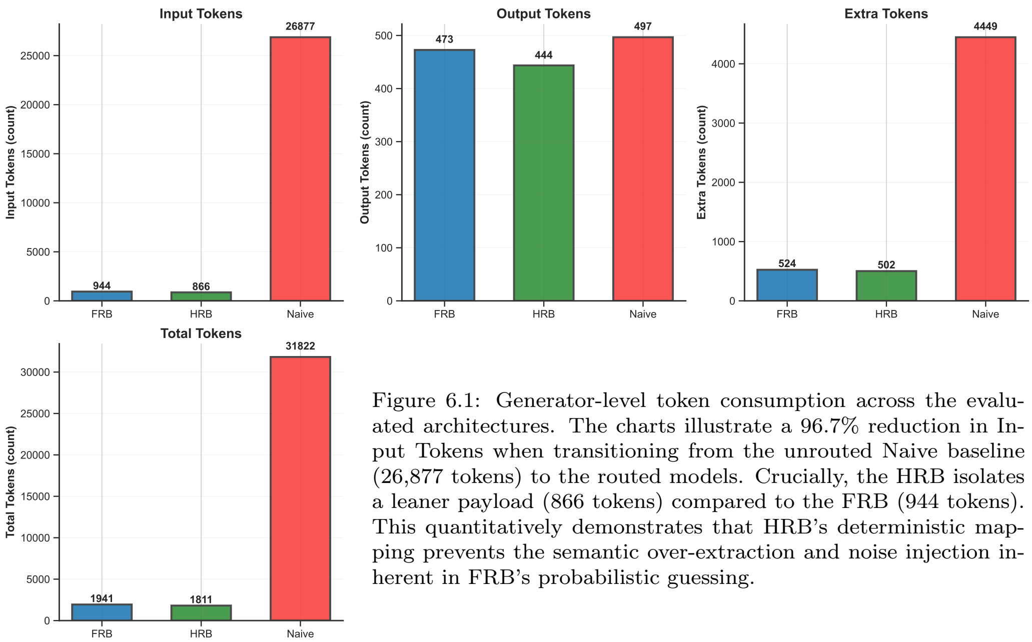


Figure 6.1: Generator-level token consumption across the evaluated architectures. The charts illustrate a 96.7% reduction in Input Tokens when transitioning from the unrouted Naive baseline (26,877 tokens) to the routed models. Crucially, the HRB isolates a leaner payload (866 tokens) compared to the FRB (944 tokens). This quantitatively demonstrates that HRB's deterministic mapping prevents the semantic over-extraction and noise injection inherent in FRB's probabilistic guessing.

Consequently, this massive reduction in both primary input and secondary recursive token loads directly translates to execution speed. As shown in Figure 6.2, the core generation latency drops from 10.18 seconds in the Naive approach to 5.94 seconds under the HRB model. The marginal latency difference between FRB (5.75s) and HRB (5.94s) confirms that the targeted structural retrieval of HRB does not incur a meaningful computational penalty during the pedagogical generation phase.

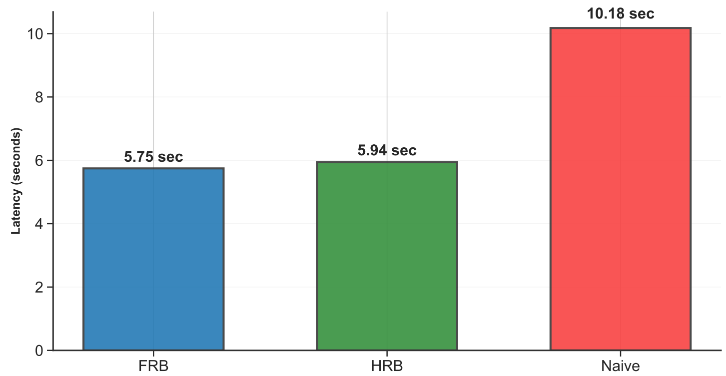


Figure 6.2: Comparison of execution times at the text-generation layer. The chart visually highlights the severe operational bottleneck of the unrouted Naive model (exceeding 10 seconds per turn). The introduction of an upstream semantic routing mechanism successfully halves the generation latency, stabilizing the user-facing response time around the 6-second threshold regardless of the specific extraction methodology (FRB or HRB).

### 6.1.2 Asynchronous Evaluation Overhead

Following the user-facing generation, the closed-loop architecture triggers the background LLM-as-a-Judge [3] to evaluate the interaction and update the database. Analyzing the telemetry of this asynchronous step exposes the compounding structural failure of the Naive baseline.

As illustrated in Figure 6.3, because the baseline lacks an intermediate filtering layer, the entire unrouted database (∼27,000 tokens) must be re-injected into the Evaluator's prompt to provide historical context for the assessment. This results in a massive 27,477 total tokens per evaluation step. While the resulting inference latency of 3.14 seconds (Figure 6.4) does not directly impact the user—being a background task—the redundant payload injection effectively doubles the API operational cost per conversational turn.

In contrast, the semantic extraction performed upstream cascades into the evaluation layer, drastically reducing the overhead. Both routed architectures process the interaction using under 1,400 total tokens, executing the required JSON extraction in approximately 1 second. This demonstrates a crucial architectural dynamic: preventing semantic bloat at the generation layer inherently supplies a cleaner, strictly relevant state vector to the Evaluator, ensuring the sustainability and financial viability of the continuous tracking loop.

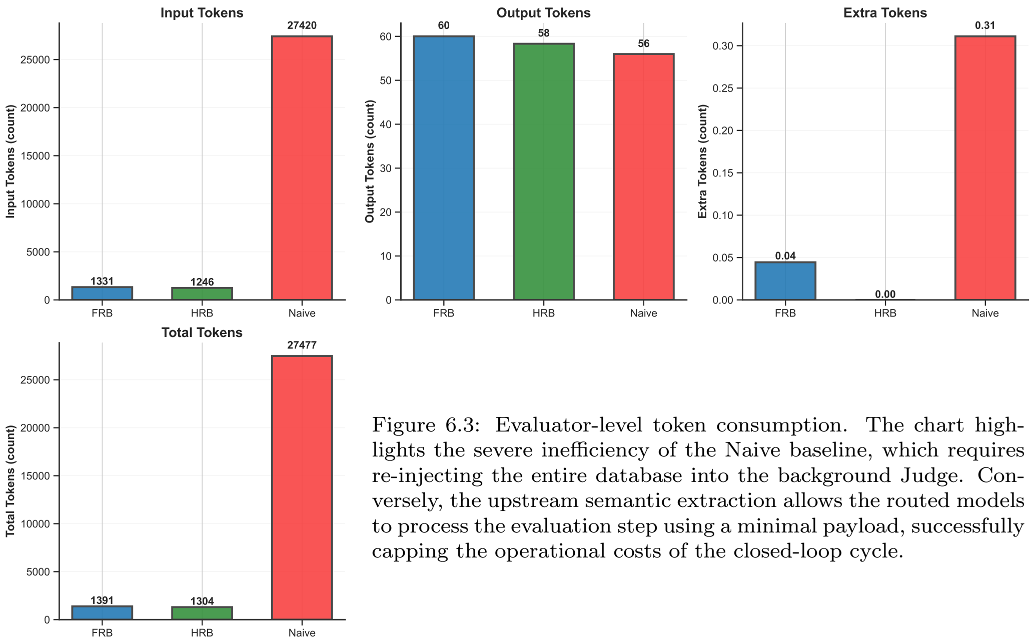


Figure 6.3: Evaluator-level token consumption. The chart highlights the severe inefficiency of the Naive baseline, which requires re-injecting the entire database into the background Judge. Conversely, the upstream semantic extraction allows the routed models to process the evaluation step using a minimal payload, successfully capping the operational costs of the closed-loop cycle.

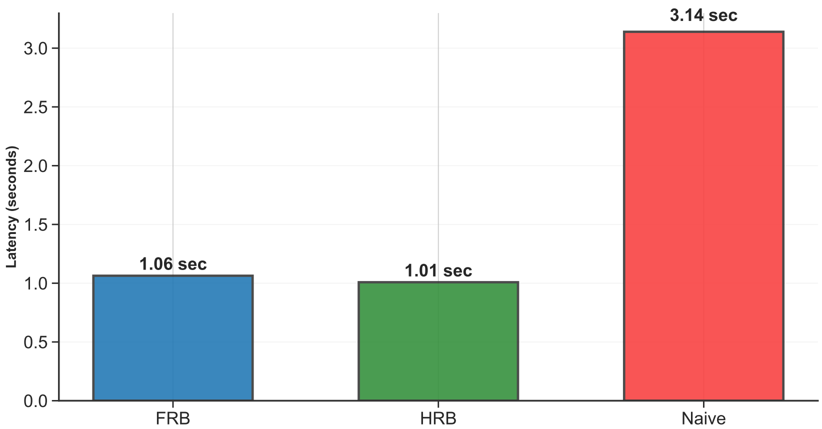


Figure 6.4: Inference latency of the asynchronous Evaluator LLM. While this background execution does not directly impact the user-facing response time, reducing the extraction process from 3.14 seconds (Naive) to approximately 1 second (Routed architectures) represents a critical backend optimization. This 3x speedup minimizes thread occupation and API concurrency bottlenecks, dramatically improving the overall scalability of the system for multi-user deployments.

### 6.1.3 End-to-End System Performance: The Architectural Trade-off

While the isolated metrics for generation and evaluation highlight the success of context filtering, a rigorous assessment requires aggregating these components to analyze the *End-to-End* (E2E) pipeline (Router + Generator + Evaluator).

As visually summarized in Figure 6.5, when accounting for the complete execution cycle, the Naive architecture exhibits a catastrophic total footprint of 59,299 tokens per single conversational turn. This makes the brute-force RAG approach financially and computationally unviable for continuous tracking.

The decision between FRB and HRB, therefore, represents a strict engineering trade-off regarding the distribution of computational load. The FRB pipeline optimizes for absolute global API cost-efficiency. By utilizing a lightweight heuristic Router, the FRB processes an E2E total of 4,094 tokens. However, its probabilistic routing causes semantic leakage, as evidenced by the slightly higher token consumption at the Generator and Evaluator stages.

Conversely, the HRB intentionally sacrifices token economy upfront to guarantee structural precision. Processing the deterministic ontology map at the routing layer inflates the E2E total to 12,553 tokens per turn. However, this upfront investment yields a highly optimized downstream pipeline, ensuring the leanest possible prompts for both the Pedagogical Tutor and the Judge (clearly visible in the compressed upper segments of the HRB bar in Figure 6.5).

Ultimately, because educational continuity relies on absolute tracking accuracy, the HRB is validated as the operational standard for this framework. It deliberately absorbs a higher computational cost at the routing layer to eliminate the semantic guesswork of the FRB, ensuring structural reliability without compromising the user-facing latency.

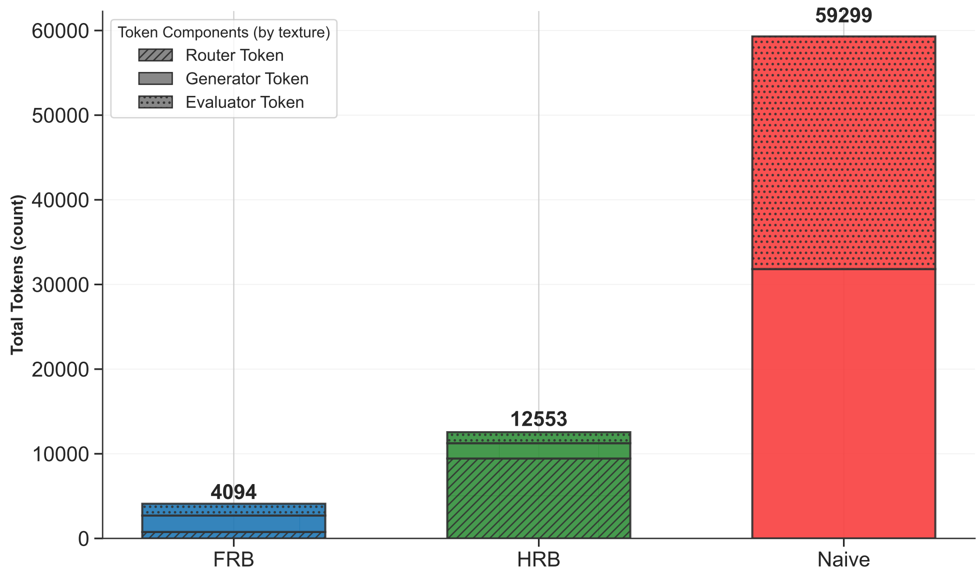


Figure 6.5: End-to-End token payload distribution across the conversational pipeline. The stacked bars illustrate the architectural trade-off: the Naive model succumbs to massive payload inflation at both generation and evaluation stages (59,299 total tokens). While FRB provides the absolute minimum global cost, the HRB intentionally allocates a larger token budget to the initial routing layer (hashed segment) to guarantee targeted, noise-free extraction for the downstream agents.

## 6.2 Qualitative Analysis of Pedagogical Interactions

While the telemetry data in Section 6.1 mathematically validates the structural efficiency of the routing protocols, a continuous educational system must ultimately be evaluated on its conversational intelligence. To assess how the varying context sizes and extraction mechanisms impact the LLM's pedagogical reasoning, an interaction audit was conducted across the three architectural models using a standardized set of user queries.

Crucially, this analysis exposes the exact *State-Tracking Vector* (the array of Topics and Subtopics) extracted by the Router and injected into the Generator LLM, providing a transparent view of the context-blinding process.

### 6.2.1 Case Study 1: Implicit Domain Alignment vs. Context Overload

A known vulnerability of unstructured RAG architectures is the "Lost in the Middle" phenomenon [20], where an LLM fed with an overwhelming amount of diverse data struggles to align its output with the implicit semantic intent of the user.

> **User Query:**
> *"Based on my tastes, what book would you recommend I read?"*
>
> ---
>
> **Naive Model** `[Context: Entire Database]`
> "Based on your extremely high interest in 'Genomic Editing (CRISPR)' and your current low knowledge in the area, I'd highly recommend reading 'A Crack in Creation' by Jennifer A. Doudna..."
>
> **HRB Model** `[Context Routed to: Literature and Writing]`
> "Considering your extremely high interest in 'Creative Writing and World-building' and 'Historical Novel vs. Magical Realism', I highly recommend diving into 'One Hundred Years of Solitude' by Gabriel García Márquez..."

**Architectural Critique:** The Naive model's response exposes the core flaw of an unrouted database when processing implicit requests. Although the user explicitly asked for a "book" (conceptually tied to the literary domain), the Naive model suffered from context overload. Having access to all academic subjects simultaneously, the LLM latched onto a completely unrelated high-interest STEM topic (CRISPR) and recommended a scientific essay. While technically grounded in the user's metadata, it missed the standard conversational intent of reading for pleasure.

Conversely, the HRB architecture successfully intercepted the implicit domain. The semantic router correctly mapped the concept of "book recommendation" to the `[Literature and Writing]` branch of the ontology. By intentionally blinding the Tutor LLM to the user's biology scores, the Router forced the generative model to stay within the appropriate domain, resulting in a perfectly aligned, genre-appropriate recommendation.

### 6.2.2 Case Study 2: The Semantic Ambiguity Trade-off (FRB vs. HRB)

Section 5.2 hypothesized that the Flat Router (FRB) operates via "probabilistic guessing" due to its lack of a hierarchical map. While token-efficient, this flat extraction makes the FRB highly vulnerable to semantic noise and keyword traps. The chat logs empirically validate this trade-off, outlining scenarios where the FRB succeeds as a low-cost alternative, and where its lack of structural awareness severely degrades the prompt payload.

**Scenario A: FRB Success (Explicit Domain Intent)**
When a user query contains explicit terminology that maps directly to a

single academic domain without distracting keywords, the FRB performs flawlessly.

**User Query:**
*"Hi, can you help me prepare for my next algebra test?"*

---

**FRB Extraction:** `[Algebra]`
**HRB Extraction:** `[Algebra]`

In this instance, both routers correctly isolated the target domain. The FRB successfully triggered the pedagogical response while consuming significantly fewer routing tokens than the HRB, validating its operational utility for straightforward queries.

**Scenario B: FRB Failure (The Keyword Trap and Cascading Semantic Noise)**
The architectural divergence strictly emerges when a query introduces cross-domain vocabulary. Without a hierarchical ontology map to anchor the primary subject, the FRB falls into the "Keyword Trap," leading to a catastrophic cascade of over-extraction at the subtopic level.

**User Query:**
*"I think the feudal system was mainly about religion. Am I right?"*

---

**FRB Extraction:**

- **Topics:** `[History, Religions and Spirituality]`
- **Subtopics:** `[Medieval History, Eastern Philosophies (Buddhism, Taoism, Hinduism), Transcendental Meditation, Norse and Celtic Mythology, Shamanism and Rites of Passage, ...]` *(12 total subtopics injected)*

---

**HRB Extraction:**

- **Topics:** `[History]`
- **Subtopics:** `[The Roman Empire: Rise and Fall, Medieval History]` *(2 total subtopics injected)*

**Architectural Critique:** This interaction perfectly illustrates the fatal flaw of flat routing architectures. The core issue here is not a failure in the LLM's semantic comprehension, but rather a mechanical limitation of the FRB's design. Because the FRB operates on a one-dimensional array of macro-topics, it correctly identified the user's explicit keyword ("religion") and triggered the `[Religions and Spirituality]` category. However, lacking a nested hierarchical structure (Topic $\rightarrow$ Subtopic) to further filter the domain, the system was forced to mechanically retrieve and dump every single subtopic tied to that category into the prompt. This structural blindness caused the indiscriminate injection of entirely irrelevant domains—such as Shamanism and Eastern Philosophies—polluting the context window with severe semantic noise.

Conversely, the HRB avoids this "Keyword Trap" because its semantic router navigates a complete topological map. It evaluates the relationships between nodes rather than just matching isolated words. It successfully anchored the core concept ("feudal system") to its correct parent node (`[History]`), recognizing the word "religion" merely as a contextual attribute of the historical inquiry. By pruning the irrelevant branches, the HRB mechanically guaranteed a precise, noise-free payload.

### 6.2.3 Case Study 3: The Proactive Recommendation Matrix

The final evaluation metric focused on the system's ability to transition from a passive answering engine to a proactive pedagogical agent. This is achieved by directly utilizing the categorical metadata labels (derived from the underlying $K$ and $I$ numerical scores) filtered by the Router.

**User Query:**
*"Hi, I have an algebra exam on Friday. What is my current situation?"*

---

**HRB Model** `[Context Routed to: Algebra]`
"Hi there! It's great that you're thinking proactively about your algebra exam on Friday. Looking at your profile, you have a high interest in Algebra, which is a huge asset! However, your knowledge in 'Algebraic expressions, Equations, and Inequalities' is currently categorized as 'Extremely low,' and you're 'Slightly lapsed' in this area. Since you have strong interest but need foundational review, I recommend starting with a quick refresher on the core principles before tackling complex problems..."

**Architectural Critique:** This interaction empirically validates the execution of the *Proactive Recommendation Matrix* and the *Semantic Discretization* methodology detailed in Sections 5.3 and 4.3.1. By injecting the pre-computed categorical labels (e.g., "Extremely low knowledge," "High interest") rather than raw numerical floats, the architecture successfully leverages the LLM's natural semantic reasoning capabilities. As demonstrated in the output, the Tutor did not suffer from numerical hallucination or interpretive ambiguity. Instead, it precisely cross-referenced the injected text labels with its systemic instructions, accurately diagnosing the user's cognitive deficit while using their high affective state to proactively formulate an encouraging, targeted study plan.

## 6.3 Evaluator Reliability and Database Integrity

While Section 6.2 demonstrated the conversational intelligence of the user-facing agents, the sustainability of the continuous tracking loop relies entirely on the background LLM-as-a-Judge [3]. To guarantee the integrity of the persistent database, this asynchronous Evaluator must accurately classify the pedagogical value of an interaction, extract precise numerical shifts, and decide when to expand the educational ontology.

Analyzing the JSON outputs from the `evaluator_logs.csv` reveals the robustness of this background agent across three critical operational dimensions.

### 6.3.1 Case Study 1: Triggering the "Kill Switch"

When a user engages in logistical or task-execution queries, the Evaluator must intervene to prevent tracking non-academic subjects as study topics.

**User Query:**
*"I'm going to London next weekend. Can you plan a 3-day itinerary focusing on museums?"*

---

**Evaluator Extraction:**
`{"is_valid_tracking_event": false}`

**Architectural Critique:** The Evaluator successfully applied the Bypass Rule. By recognizing the interaction as a "Task/Execution Event" rather than a pedagogical "Learning Event," it immediately halted the extraction process. This preemptive block prevents a database write, preserving the purity of the user's academic state vectors.

### 6.3.2 Case Study 2: Nuanced Metric Extraction

When a valid pedagogical event occurs, the Judge must evaluate the cognitive ($K$) and affective ($I$) state changes without hallucinating arbitrary numbers. To demonstrate the system's dynamic range, two contrasting interactions were analyzed: one indicating cognitive struggle, and another indicating academic mastery.

**User Query (Struggle & Frustration):**
*"I keep failing the mock tests on organic chemistry nomenclature. I just don't get the IUPAC rules and I'm getting really tired of it."*

---

**Evaluator Extraction:**


```
{
"is_valid_tracking_event": true,
"is_new_topic": false,
"topic": "Chemistry and Biochemistry",
"subtopic": "Organic Chemistry and Drug Synthesis",
"interaction_knowledge": 12.0,
"interaction_interest": 8.0
}
```


**Architectural Critique:** This log highlights the advanced semantic comprehension of the evaluator prompt. Instead of defaulting to a baseline score, the system accurately quantified the user's explicit admission of failure (*Knowledge: 12.0*) and their expressed frustration (*Interest: 8.0*). These precise numerical weights drag the Exponential Moving Average (EMA) downward, ensuring the system registers the user's struggle accurately and adapts the subsequent pedagogical approach.

**User Query (Mastery & Eagerness):**
*"I'm reviewing for my calculus final. I'm pretty solid on basic integrals, so let's jump straight into advanced integration by parts. Give me a hard problem."*

---

**Evaluator Extraction:**


```
{
"is_valid_tracking_event": true,
"is_new_topic": false,
"topic": "Integration",
"subtopic": "Integration Techniques",
"interaction_knowledge": 90.0,
"interaction_interest": 90.0
}
```


**Architectural Critique:** Conversely, this interaction demonstrates the Evaluator's capacity to recognize academic confidence and proactive engagement. The user explicitly states their competence ("pretty solid") and directly challenges the system ("Give me a hard problem"). The LLM-as-a-Judge [3] correctly assigns extremely high scores in both dimensions (*Knowledge: 90.0, Interest: 90.0*). This sharply shifts the user's state-tracking vector into the upper percentiles, triggering the Generator's Recommendation Matrix to skip foundational explanations and immediately provide advanced, complex tasks in future interactions.

### 6.3.3 Case Study 3: Dynamic Ontology Expansion

The framework is designed to reuse existing topics (Greedy Matching) to prevent database fragmentation. However, when confronted with a radically new discipline, it must autonomously expand the educational ontology.

**User Query:**
*"Explain the tactical differences between a flanking maneuver and a double envelopment in modern combined arms warfare."*

---

**Evaluator Extraction:**


```
{
"is_valid_tracking_event": true,
"is_new_topic": true,
```

```
"topic": "Military Tactics",
"subtopic": "Tactical Maneuvers",
"interaction_knowledge": 60.0,
"interaction_interest": 85.0
}
```

**Architectural Critique:** Lacking any existing categories related to military strategy in the database, the Evaluator correctly abandoned the Greedy Matching protocol and triggered the Open Discovery rule (`is_new_topic: true`). It autonomously generated a formally correct, hierarchical pairing (`[Military Tactics]` → `[Tactical Maneuvers]`), seamlessly expanding the knowledge graph without requiring manual developer intervention.

# Chapter 7

# Conclusions

## 7.1 Summary of Findings

The primary objective of this thesis was to address the inherent statelessness of foundational Large Language Models (LLMs) and engineer a scalable, memory-aware architecture capable of continuous user personalization. Set against the backdrop of the European DataGEMS project [1] and the MathE educational platform [2], this research successfully transitioned a generic text-generation model into a proactive, context-aware pedagogical agent.

Through the development and evaluation of the *Closed-Loop Hierarchical Router-Based (HRB)* architecture, several key findings emerged:

1. **Resolution of Context Window Exhaustion:** The empirical telemetry demonstrated that standard, unrouted Retrieval-Augmented Generation (RAG) is computationally unviable for continuous tracking, inflating prompt payloads to almost 60,000 tokens per turn (as detailed in Figure 6.5). The implementation of the hierarchical semantic router (HRB Model) successfully solved this bottleneck, compressing the generative payload while simultaneously preventing the semantic noise and over-extraction vulnerabilities associated with probabilistic Flat Routing (FRB).

2. **Autonomous and Continuous State Tracking:** The research validated the *LLM-as-a-Judge* [3] paradigm as a highly effective mechanism for asynchronous background telemetry. By deploying a secondary LLM to extract numerical cognitive ($K$, Lapse) and affective ($I$) shifts from unstructured conversational logs, the system established an autonomous closed loop. This entirely eliminated the need for intrusive, explicit user questionnaires, proving that short-term conversational interactions contain sufficient semantic signals to update a persistent,

longitudinal user profile via Exponential Moving Average (EMA) and asynchronous decay functions.

3. **Mitigation of Numerical Hallucination:** A critical finding regarding human-AI interaction was the necessity of *Semantic Discretization*. The architecture proved that forcing an LLM to interpret raw numerical floats degrades pedagogical reliability. By mapping mathematical data into discrete categorical labels (e.g., "Extremely Lapsed," "High Interest"), the system successfully leveraged the LLM's natural semantic reasoning capabilities, enabling the precise execution of the Proactive Recommendation Matrix.

4. **Dynamic Database Integrity:** Finally, the engineering of strict Semantic Constraining within the Evaluator prompt successfully safeguarded the system's database. The implementation of a "Kill Switch" prevented the tracking of meta-intents and logistical queries, while the "Open Discovery Strategy" allowed the system to seamlessly expand its ontology to include Out-of-Domain topics dynamically introduced by the user, without requiring manual developer intervention.

Ultimately, this thesis proves that by layering a deterministic hierarchical router over an asynchronous semantic evaluator, it is possible to achieve highly personalized, state-aware AI interactions that remain token-efficient, pedagogically reliable, and computationally sustainable over extended lifecycles.

## 7.2 Strengths and Limitations of the Approach

The proposed architecture demonstrates significant strengths in transforming passive Retrieval-Augmented Generation into an active, state-aware framework. Its modularity allows the generation and evaluation layers to operate asynchronously, preserving user-facing latency. Furthermore, the integration of Semantic Discretization successfully bridges the gap between quantitative learning analytics and natural language generation.

However, the transition from deterministic algorithms to generative agents introduces several inherent constraints and architectural limitations that must be critically acknowledged:

- **The Global Profile Query Dilemma:** The most prominent structural limitation of the HRB framework is its mathematical inability to process holistic meta-queries. Because the semantic router is explicitly engineered for strict *local extraction* (isolating a single topic to minimize the context window), it inherently fails when a user asks a global question such as, *"Based on my entire history, what subject am I worst at?"* In these instances, the HRB fails to match a specific knowledge domain and passes an empty (or incomplete) array to the Generator. While the Naive model is technically capable of answering these global queries by reading the entire database natively, this brute-force approach is practically unviable due to severe context overload, token exhaustion, and a high risk of semantic hallucinations. Consequently, resolving this dilemma efficiently within the HRB architecture requires the future implementation of an upstream Intent Classifier to trigger a deterministic, metric-driven routing protocol, rather than reverting to massive database injections.

- **API Dependency and Scalability Costs:** While the HRB architecture drastically reduces the catastrophic token consumption of the Naive baseline at the generation layer, it remains intrinsically dependent on proprietary LLM APIs (the Gemini 2.5 [19]). The HRB Router requires injecting the full hierarchical ontology map at every turn (∼9,400 tokens). In an enterprise-scale deployment like the MathE platform, supporting thousands of concurrent students, this upfront routing cost constitutes a substantial operational bottleneck and introduces a strict vendor lock-in, making the system vulnerable to third-party pricing changes and API rate limits.

- **Prompt Fragility and Model Sensitivity:** The integrity of the continuous tracking loop relies heavily on strict prompt engineering within the background Evaluator. While effective for this proof of concept, relying on natural language instructions to enforce deterministic JSON schema outputs is a fragile software engineering pattern. The system's evaluation accuracy is tightly coupled to the specific semantic latent space of `gemini-2.5-flash-lite`. A future model deprecation or a migration to a different foundational model (e.g., Llama 3 or GPT-4o) would alter the Evaluator's sensitivity, requiring extensive recalibration to prevent the "Kill Switch" or the "Open Discovery" rules from failing.

- **Intrinsic Hallucination Risks and Ontological Drift:** Finally, despite the implementation of Semantic Constraining and Greedy Matching, the system relies on probabilistic generative models. There remains a non-zero risk of "Ontological Drift"—where the Evaluator misclassifies a query and pollutes the database with an improperly generated, redundant subtopic. Similarly, the Generator LLM remains susceptible to pedagogical hallucinations, occasionally inventing mathematical concepts despite being accurately grounded by the state-tracking vector. Absolute mitigation of these risks is impossible within purely generative, non-symbolic architectures.

## 7.3 Future Directions

While the Closed-Loop HRB architecture establishes a robust proof of concept for memory-aware educational agents, its current implementation serves as a foundational baseline. To fully realize the vision of the DataGEMS project and scale the framework for enterprise-level deployments like the MathE platform, several critical advancements are proposed for future work:

- **Upstream Intent Classification for Meta-Queries:** To resolve the architectural limitation surrounding Global Profile Queries, future iterations must implement an ultra-lightweight Intent Classifier (e.g., a fine-tuned BERT model [21] or a low-latency LLM call) positioned immediately before the HRB layer. This classifier will act as a semantic switch: routing standard academic questions to the HRB for semantic extraction, while intercepting meta-queries ("What should I review today?") to trigger a deterministic, Reverse Metric-Driven filter directly on the database. This hybrid routing will guarantee comprehensive query coverage without inflating token payloads.

- **Vector Databases for Enterprise-Scale Semantic Retrieval:** Currently, the HRB architecture relies on mapping text-based ontologies from a relational tabular repository (`context_data.csv`). As the student's longitudinal profile expands over a multi-year academic degree, maintaining and evaluating even a localized hierarchical map could eventually strain the token payload. To achieve true industrial scalability, future iterations should migrate the persistent profile into a dedicated Vector Database. By converting the State-Tracking Vectors into high-dimensional embeddings, the system can leverage Dense Retrieval [22] (e.g., Approximate Nearest Neighbor algorithms [8] ) to extract only the Top-$K$ most semantically relevant historical states. This transition from static hierarchical routing to dynamic embedding-based similarity search aligns with current enterprise RAG methodologies, ensuring that token consumption remains strictly capped and computationally viable regardless of the underlying database volume.

- **Multimodal Context Integration:** Modern foundational models, including the Gemini family [19] utilized in this research, possess advanced multimodal capabilities. A logical progression for this architecture is the integration of visual data processing. Allowing students to upload images of handwritten equations, erroneous test papers, or geometric diagrams—and appending this visual context to the active State-Tracking Vector—would dramatically enhance the Tutor's diagnostic precision and pedagogical utility.

- **Longitudinal Validation with Human-in-the-Loop:** The empirical evaluations in this study heavily relied on telemetry benchmarking and simulated interaction logs augmented from the MathE dataset. The definitive validation of the Closed-Loop architecture requires deployment in a live, real-world educational environment. Future work must encompass longitudinal A/B testing with active university students over a full academic semester. This will allow researchers to measure the actual impact of the Proactive Recommendation Matrix on quantifiable educational outcomes, such as standardized test scores, student retention rates, and long-term engagement metrics.

In conclusion, the transition from stateless foundational models to state-aware, personalized cognitive engines represents the next frontier in Human-Computer Interaction. The hierarchical routing and asynchronous evaluation mechanisms developed in this thesis provide a mathematically sound, token-efficient blueprint for building AI systems that do not merely answer questions, but actively learn how to teach.

# Bibliography

[1] European Commission. DataGEMS: Data Discovery Platform with Generalized Exploratory, Management, and Search Capabilities. Horizon Europe Research and Innovation Programme, Grant Agreement No. 101188416, 2025. `https://datagems.eu/`.

[2] MathE Project. Mathe: Improving the learning of mathematics in engineering. `https://mathe.pixel-online.org/`, 2026. Accessed: 2026-06-07.

[3] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Yannis Ioannou, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.

[5] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.

[6] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, Lei Li, and Zhifang Duan. A survey for in-context learning. *arXiv preprint arXiv:2301.00234*, 2022.

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[8] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[9] Albert T Corbett and John R Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction*, 4(4):253–278, 1994.

[10] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in neural information processing systems (NeurIPS)*, volume 28, 2015.

[11] Hermann Ebbinghaus. *Memory: A contribution to experimental psychology*. Duncker & Humblot, 1885.

[12] Rosalind W Picard. *Affective Computing*. MIT press, 1997.

[13] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.

[14] Jinheon Baek, Nirupama Chandrasekaran, Allen Herring, and Silviu Cucerzan. Knowledge-augmented large language models for personalized contextual query suggestion. In *Proceedings of the ACM on Web Conference 2024*, pages 3366–3377. ACM, 2024.

[15] Mengjuan Li and Lei Niu. Exercise recommendation with preferences and expectations based on ability computation. *Computers, Materials & Continua*, 77(1), 2023.

[16] Ge Gao, Amelia Leon, Andrea Jetten, Jasmine Turner, Husni Almoubayyed, and Stephen Fancsali. Predicting long-term student outcomes from short-term edtech log data. In *Proceedings of the 15th International Learning Analytics and Knowledge Conference (LAK 25)*. ACM, 2025.

[17] Tencent. TencentDB Agent Memory: A 4-Tier Local Memory Pipeline for AI Agents. GitHub repository, 2026. `https://github.com/Tencent/TencentDB-Agent-Memory`.

[18] Robert A. Bjork and Elizabeth L. Bjork. A new theory of disuse and an old theory of stimulus fluctuation. In A. F. Healy, S. M. Kosslyn, and R. M. Shiffrin, editors, *From Learning Processes to Cognitive Processes: Essays in Honor of William K. Estes, Vol. 2*, pages 35–67. Erlbaum, Hillsdale, NJ, 1992.

[19] Google Gemini Team. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[20] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[21] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 2019.

[22] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

# Ringraziamenti

Quando mi sono iscritto a questo corso, non sapevo se fosse la scelta giusta: dopo la triennale la voglia di studiare era poca e gli ultimi esami mi avevano lasciato piuttosto provato. Tuttavia, grazie anche alla spinta dei miei genitori, che hanno sempre creduto in me, ho deciso di procedere. A loro va il mio primo e più grande ringraziamento: mi hanno spronato a proseguire il mio percorso di studi e mi hanno sostenuto anche nelle scelte più difficili e azzardate, come quella di lasciare un lavoro che non mi soddisfaceva per tornare a concentrarmi maggiormente su quest'ultimo anno di magistrale.

Un altro ringraziamento speciale va ad Elisabetta, che mi è sempre stata vicina, supportandomi con grande entusiasmo e positività. La tua carica mi ha dato la motivazione necessaria per affrontare ogni sfida con lo spirito giusto. Grazie anche per aver sopportato il mio stile di vita da studente universitario in questo ultimo anno!

Un grazie speciale va anche ai miei amici, con i quali sono cresciuto e con cui, ancora oggi, trascorro gran parte del mio tempo. Stare con voi mi permette di staccare la spina: i nostri confronti e i nostri progetti sono per me uno stimolo costante a crescere e migliorare insieme.

Un sentito ringraziamento va al mio relatore, il prof. Niccolò Marastoni: il suo supporto e le sue preziose indicazioni sono stati fondamentali per lo sviluppo di questa tesi. Inoltre, il suo corso del primo anno, oltre ad aver consolidato il mio interesse verso questo ambito, mi ha permesso di comprendere appieno le prospettive e le potenzialità del corso di laurea, offrendomi la spinta necessaria per proseguire con determinazione questo percorso.

Infine, un ultimo ringraziamento va a me stesso. Quando mi sono iscritto a questa magistrale, sapevo che l'avrei portata a termine, anche a costo di impiegarci quindici anni; alla fine ne sono bastati tre e devo dire che mi sono tolto più di qualche soddisfazione. Se c'è una cosa che questo percorso mi ha insegnato è che i sacrifici spesso ripagano e che avere fiducia nelle proprie capacità, insieme alla volontà di mettersi in gioco, è la chiave per aprire molte porte.

Grazie mille, ci vediamo nel percorso.

Andrea